Latest updates: https://dl.acm.org/doi/10.1145/3696630.3731665

RESEARCH-ARTICLE

The Evolution of Information Seeking in Software Development: Understanding the Role and Impact of AI Assistants

EBTESAM AL HAQUE, George Mason University, Fairfax, VA, United States

CHRISTOPHER BROWN, Virginia Polytechnic Institute and State University, Blacksburg, VA, United States

THOMAS D. LATOZA, George Mason University, Fairfax, VA, United States

BRITTANY JOHNSON, George Mason University, Fairfax, VA, United States

Open Access Support provided by:

George Mason University

Virginia Polytechnic Institute and State University

PDF Download
3696630.3731665.pdf
11 February 2026
Total Citations: 1
Total Downloads: 690

Published: 23 June 2025

Citation in BibTeX format

FSE Companion '25: 33rd ACM International Conference on the Foundations of Software Engineering
June 23 - 28, 2025
Trondheim, Norway

Conference Sponsors:
SIGSOFT

FSE Companion '25: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (June 2025)
https://doi.org/10.1145/3696630.3731665
ISBN: 9798400712760

The Evolution of Information Seeking in Software Development: Understanding the Role and Impact of AI Assistants

Ebtesam Al Haque
George Mason University
Fairfax, Virginia, USA
ehaque4@gmu.edu

Chris Brown
dcbrown@vt.edu
Virginia Tech
Blacksburg, Virginia, USA

Thomas D. LaToza
George Mason University
Fairfax, Virginia, USA
tlatoza@gmu.edu

Brittany Johnson
George Mason University
Fairfax, Virginia, USA
johnsonb@gmu.edu

Abstract

About 32% of a software practitioners' day involves seeking and using information to support task completion. Although the information needs of software practitioners have been studied extensively, the impact of AI-assisted tools on their needs and information-seeking behaviors remains largely unexplored. To addresses this gap, we conducted a mixed-method study to understand AI-assisted information seeking behavior of practitioners and its impact on their perceived productivity and skill development. We found that developers are increasingly using AI tools to support their information seeking, citing increased efficiency as a key benefit. Our findings also amplify caveats that come with effectively using AI tools for information seeking, especially for learning and skill development, such as the importance of foundational developer knowledge that can guide and inform the information provided by AI tools. Our efforts have implications for the effective integration of AI tools into developer workflows as information retrieval systems and learning aids.

CCS Concepts

• **Human-centered computing** → **Empirical studies in HCI**; • **Software and its engineering** → *Programming teams.*

Keywords

software engineering, developers, mixed-methods study, ai, productivity, information seeking, skill building

ACM Reference Format:
Ebtesam Al Haque, Chris Brown, Thomas D. LaToza, and Brittany Johnson. 2025. The Evolution of Information Seeking in Software Development: Understanding the Role and Impact of AI Assistants. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3696630.3731665

This work is licensed under a Creative Commons Attribution 4.0 International License.
FSE Companion '25, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3731665

1 Introduction

Artificial Intelligence (AI) has revolutionized how we think about software engineering. According to the most recent StackOverflow Developer Survey, 76.7% of developers are already using or plan to incorporate AI-assisted software development assistants into their workflows [1]. Developers use AI-assisted software tools in a variety of contexts and to complete a diversity of tasks, most often hoping for increases in their productivity [14].

A significant contributor to, or deterrent from, developer productivity is the time they spend seeking information to support the completion of their tasks [19, 31]. Over the years, we have acquired an in-depth understanding of the information needs developers have [24, 29, 34], the ways in which they attempt to meet their information needs [18], and the challenges they encounter in the process [25]. We also know that developers' ability to meet information needs contributes to their ability to acquire and build expertise [5, 17].

Given the changing landscape in developer tooling, many studies have investigated the use and impact of AI-assisted tools on developer engagement and productivity [12, 13, 37, 40]. However, most of these studies focus on AI-assisted tool use in the context of understanding, generating, or modifying source code, providing little insight into the role AI plays in developer information seeking and its impact. To address this gap, we conducted a mixed methods study to answer the following research questions:

RQ1 When, why, and how do developers use AI tools for information seeking?
RQ2 What impact does the use of AI tools for information seeking have on developer productivity?
RQ3 What impact does the use of AI tools for information seeking have on developer skill development?

To answer our research questions, we first administered a survey (n = 128) to better understand to what extent developers engage with AI tools in this context. Building on those insights, we conducted a series of 17 interviews to further contextualize participants' experiences. Our findings reveal that while AI tools can offer increased efficiency when seeking information, there are caveats to realizing the benefits, such as avoiding over-reliance and building the necessary expertise to appropriately and effectively use AI tooling to meet information needs.

2 Related Work

The goal of our research is better understand information seeking with the availability of AI-assisted tools. Most relevant to our efforts are investigations into the information needs and seeking behaviors of developers and human-centric concerns regarding the use of AI tools in software development.

2.1 Information Needs of Developers

Previous studies have examined the information needs of developers during the software development process. One of the first efforts centered on developer information needs was conducted by Ko *et al.* [24], where they observed developers in their work to better understand information needs of collocated software teams. Also aiming to gain a broad understanding of developer information needs is the work done by Fritz *et al.* [18], who conducted a series of 11 interviews to explore the use of information fragments to answer developers' questions. On the flip side, LaToza *et al.* [25] surveyed 179 professional developers regarding hard-to-answer questions about code to better understand information needs developers encounter challenges trying to fulfill.

Some prior efforts are more focused, with an interest in understanding specific information needs. Breu *et al.* [10] analyzed information needs in bug reports by examining questions asked in six hundred bug reports from the Mozilla and Eclipse projects. Liu *et al.* [29] investigated API-related developer information needs on Stack Overflow by annotating Stack Overflow questions to APIs. Phillips *et al.* [34] explored information needs for integration decisions in the release process of large-scale parallel development by interviewing seven release managers. With the emergence of AI-assisted tools, our builds on prior efforts by providing insights into the ways in which these tools have changed software practitioners' information-seeking behaviors.

2.2 AI Tool Use in Software Engineering

Russo [36] identified factors that influence adoption of Generative AI tools in software engineering and found that compatibility with existing development workflows was the primary driver of AI tool adoption, rather than perceived usefulness or social factors. Others have studied the context and impact of integrating AI tools into software engineering processes, some of which we discuss below. Barke *et al.* [4] investigated how developers interact with AI programming assistants like GitHub Copilot. They classified interaction modes into two types: *acceleration mode*, where developers know the next steps and use Copilot to speed up their work, and *exploration mode*, where developers are unsure and use Copilot to discover possible solutions. Bird et al. [6] explored early user experiences with Copilot in pair-programming contexts, revealing that while the tool can provide useful initial code suggestions, it often requires additional review, hence shifting some of the workload from coding to code validation. Liang et al. [28] conducted a large-scale survey to investigate the usability of AI programming assistants, focusing on the successes and challenges developers face when using these tools.

Johnson *et al.* [22] developed the PICSE framework, which identifies key factors influencing engineers' trust and usage of traditional and AI-assisted software tools. Their framework was derived from interviews with software practitioners.

Given the concern regarding the impact of AI-assisted tool use on productivity, prior work has also investigated how we can measure developer productivity when using code completion tools like GitHub Copilot1[40].

Our work builds on these prior efforts by providing insights into how these developer interactions with AI tools influence not just task completion, but also their information-seeking behaviors and its impact on their productivity and skill development.

3 Methodology

The goal of our study is to better understand information seeking behaviors when using AI tools to complete software development tasks. Below we describe the study we conducted towards this goal.

3.1 The Survey

We designed a 20-minute survey administered through Qualtrics to engage developers in our research.[1] Our survey was divided into four sections. The first section asked a series of demographic and background questions, such as current job role and years of programming experience. The design of this section was heavily influenced by the annual Stack Overflow Developer Survey [1]. The next section asked questions about the specific AI-assisted tools they use, the frequency and purpose of use, and rationale behind their usage. We also asked questions about their information seeking behaviors with and without AI assistance, including the advantages and disadvantages for each. To assess the impact of AI tools on development tasks, we also asked respondents to evaluate how these tools influenced their approach to tasks like debugging, testing, implementing, and planning. In the final section of the survey, we included questions regarding the role of AI tool in learning and integrating new technologies, such as the impact on learning curves and the potential pitfalls. We also gave respondents the opportunity at the end of the survey to express interest in a follow-up interview (Section 3.2). Prior to administering our survey, we piloted with 3 respondents. This helped ensure clarity and identify any issues before deployment. We excluded this data from our final dataset. *The survey instrument is available in our supplemental materials [2].*

Our goal was to recruit developers with various levels of experience, technical skills, job roles, and team sizes. Therefore, we advertised our survey in both virtual and physical settings. We promoted our efforts through the personal LinkedIn and Twitter accounts of all authors, as well as through internal mailing lists. We also reached out to developer communities in the Washington D.C. Metropolitan area, where most of the authors are based. We also used snowball sampling by encouraging participants to share the survey with their networks. In total, we received 310 responses, with 173 participants expressing interest in participating in a follow-up interview.

Data Preparation

Given that we distributed our survey through social platforms, there is a heightened risk of receiving invalid responses [20]. To

[1]This research is approved under IRBNet #: 2163056-1.

mitigate the potential for analyzing invalid data, we filtered out responses that were completed in under three minutes or incomplete. This initial filtering left us with 168 responses. We further excluded responses that contained irrelevant content in their open-ended answers, as this suggests they may not have been giving due consideration to the survey. This resulted in our final dataset, containing 128 valid responses, which we used to report our findings.

Table 1: Age Groups of Survey Respondents

Age Group	*No. Respondents*
18-24 years old	44
25-34 years old	73
35-44 years old	6
45-54 years old	3
55-64 years old	2
65 years or older	1

Table 2: Professional Programming Experience Among Survey Respondents

Years of Experience	*No. Respondents*
0-2 years	36
2-5 years	53
5-7 years	21
7-10 years	10
More than 10 years	8

Table 3: Job Titles of Survey Respondents

Job Title	*No. Respondents*
Developer (full stack, frontend or backend)	62
Research & Development Role	15
Data Scientist or Machine Learning Specialist	14
Data or Business Analyst	13
Project Manager	5
DevOps Specialist	4
Security Professional	4
Other	3
Designer	3
Senior Executive (C-suite, VP, etc.)	2
QA or Test Engineer	1
System Administrator	1
Cloud Infrastructure Engineer	1

Respondents

In our final dataset of responses (128), the age of our survey respondents ranged from 18 to over 65, median being 25-34 [Table 1] with of 4 years of experience (Table 2). Prior research suggests that developers with less experience spend more time seeking information and learning as they transition from novices to experts [5] [17] [27], making this demographic particularly valuable for our study. Majority of our survey respondents actively use AI tools (83.6%) such as ChatGPT and GitHub Copilot (Table 4) in their work, while 16.4% occasionally use them. In addition to formal education, a majority of our survey respondents also used books and online resources such as forums and courses to learn programming. We interviewed a subset of these respondents, depending on their interest and availability and report on their demographics in Table 5.

Table 4: AI-Assisted Tools Used by Survey Respondents

Tool	*No. Respondents*
ChatGPT	118
GitHub Copilot	62
Gemini	51
Microsoft Copilot	38
Tabnine	8
Claude	4
Other	8

Data Analysis

To analyze our survey, we first mapped each survey question to our research questions. We categorized experience levels of participants into five different groups, as outlined in Table 2. For closed ended questions, we ran descriptive statistics and applied Fisher's exact test with Bonferroni correction to identify any correlations. We also report on frequencies to supplement our findings. For open-ended responses, we used thematic summaries to report findings due to the small sample size of valid and useful responses. We only report significant findings in Section 4 .

3.2 The Interviews

To supplement our survey findings, we conducted interviews over Zoom to gather detailed insights into participants' information-seeking practices, experiences with AI-assisted tools, and their impact on productivity and skill development. This helped us obtain a variety of perspectives and allowed for immediate follow-up questions for clarification.

Each interview began with a discussion of participants' background in software development, their experiences with AI-assisted tools, and how these tools fit into their workflow. Topics covered included the impact of AI tools on productivity, skill development, and problem-solving strategies. We also asked questions regarding their roles and responsibilities, the typical activities in their jobs, and their reliance on other developers or software tools. This includes what AI tools they use, or avoid use of, when information seeking and their satisfaction with the interactions. We elicited specific examples of their use to better understand aspects such as workflow integration and problem solving approaches. Lastly, we inquired about the impact team dynamics and productivity.

We piloted the interview protocol following best practices with one author and two participants to identify and correct any issues. *The final interview protocol is available in our supplemental materials [2].* We continued interviewing participants until we reached theoretical saturation, where no new themes or insights emerged

from the interview [21]. Table 5 details the background of our interview participants.

Table 5: Interview Participant Background

Participant	*Years of Experience*	*Job Title*
P1	2.5	Software Developer
P2	3	Software Developer
P3	2	Technical Analyst
P4	3.5	Data Analyst
P5	3	Software Developer
P6	2.5	Software Developer
P7	1	Business Analyst
P8	1.5	Software Developer
P9	3	Software Developer
P10	0.5*	Software Developer
P11	4	Research Engineer
P12	1.5	Data Engineer
P13	2.5	Software Developer
P14	2	Software Developer
P15	5	Software Developer
P16	6	Software Developer
P17	18	Software Engineering Lead

* has background in instructional design and writing data analysis software prior to current role

Analysis

To analyze our interview data, we used qualitative coding and thematic analysis, beginning with the creation of an initial codebook based on questions asked during the participant interviews. In the first iteration, all four authors independently labeled an interview transcript using the initial codebook and then collectively discussed their findings, which led to the emergence of a new set of codes. After agreement on the codes and how they should be used, we incorporated the emergent codes to finalize the codebook. Following this, we applied the final codebook [2] to label the remaining transcripts.

To ensure the rigor and validity of our efforts, we invited an external auditor to review a subset of raw coded segments from the interview transcripts along with the codebook. We asked the auditor to confirm that the inferences we made from the data were sound. We discussed and clarified any concerns and updated code descriptions when deemed necessary. After validating the codes, the first author reviewed all coded segments under each code and documented the overarching themes as they emerged. Finally, we mapped the codes to our research questions and iteratively identified any overlapping themes.

4 Results

The goal of our efforts are to uncover when, why, and how developers are using AI tools to support their information seeking, and their impact on developer productivity and skill development. Below we discuss our findings regarding the considerations developers may make when deciding to engage with AI tools to meet their information needs.

4.1 Information Seeking with AI tools (RQ1)

AI-Assisted Information Seeking

Our findings suggest that developers are often using AI tools to support their information seeking Figure 1), with the majority reporting using AI tools at least half of the time in their work. However, according to the experiences reported in our interviews, their goals when doing so are most often broad and aimed at understanding the necessary considerations for completing their task.

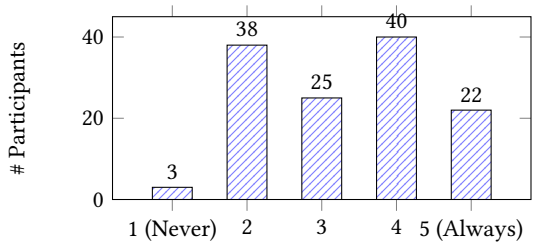

Figure 1: Frequency of AI Use for Information Seeking

Most participants reported relying on AI tools when trying to **understand best practices**, **discover new libraries** or solutions or **explore trade-offs** between different libraries and implementations.

Participants in our study also cited AI tools as being useful for **identifying keywords for further online search** and validation.

Another common use case among participants when information seeking is to use AI tools to **recall previous knowledge** or **explain code**. Participants mentioned value in using AI tools for synthesizing relevant information from documentation, which includes providing boilerplate code, pinpointing highly specific issues or information. One participant shared a concrete example of this

> *"I can find pretty much any niche thing I need to know about AWS, whereas before, it required sifting through extensive documentation and numerous Amazon support pages."[P15]*

UX Issues in AI-assisted Information Seeking

While participants in our study outlined numerous benefits to AI-assisted information seeking, concerns surfaced regarding the usability of AI tools. Most prominent were discussions regarding the **non-prescriptive language** used by AI tools. Rather than identifying critical requirements as mandatory, AI tools often present information using deferential suggestions that can undermine technical imperatives, as one participant explained:

> *"For something like code review, it's a bit riskier because error handling is not just a nice-to-have; it's a must-have. While the AI correctly identifies many issues, it frames them as improvements rather than necessities."[P15]*

Participants also highlighted how AI's adaptive responses create unique challenges for technical information validation. Unlike static documentation or community forums where information remains consistent, participants observed that AI tools can shift their technical guidance based on user interactions: *"It provided the correct information initially but later changed its stance to align with what I*

was saying. So now I don't know when to trust it." This inconsistency particularly undermines AI tools' utility as authoritative technical references. Additionally, participants reported issues with **inappropriate information density**, noting that *"Sometimes they tend to give out too much information, sometimes too little,"* making it difficult to efficiently extract needed knowledge. We discuss more about these impacts on learning new technologies in Section 4.3.

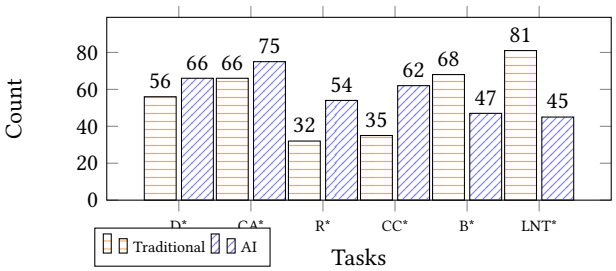

Figure 2: Traditional vs. AI preference for tasks

Task Legend: D* = Debugging, CA* = Code Analysis, R* = Refactoring, CC* = Code Comprehension, B* = Brainstorming, LNT* = Learning New Technologies.

AI-Generated Information Validation Process

Our findings reveal a contrast between traditional and AI-based information validation. Traditional sources rely on established mechanisms such as community vetting—creating standardized validation pathways. AI-generated content, however, lacks such mechanisms. Participants described not only the burden of validation but also the challenge of incomplete information. One participant's experience illustrates this problem: *"ChatGPT provided a partial answer that it scraped from Python docs regarding ArgParse library. I had to go to the actual docs to find it had a lot of missing context and prior steps."* This fragmented information delivery requires users to fill substantial knowledge gaps. Many employ cross-referencing between multiple AI systems to identify conflicts and inconsistencies, as one explained: *it's especially helpful to use multiple AI assistants"* to *find a whole pile of issues [...] Hallucinations don't seem to compound.".* The efficiency gains promised by AI-assisted information seeking are partially offset by these added validation demands, a challenge that current AI interfaces do little to address.

Traditional Information Seeking

When discussing their use of AI tools, many participants described tasks, methods, and tools they rely on for information seeking when AI falls short. Many survey respondents indicated that AI tools are more useful than traditional methods for a number of tasks (Figure 2). However, interview participants often discussed the need for human-centric resources when unable to resolve the issue or complete the task with AI-generated information alone.

> *"So if I'm stuck, I go to other colleagues, but then, before going to my colleague or mentors for help, [I've been] using this [AI tool] as help to solve the problem."[P13]*

The most common resource mentioned for information seeking without AI assistance was Stack Overflow, which participants cited using for **context-specific tasks**, such as resolving environment-specific issues, in the hopes that other developers have encountered similar scenarios. For **GUI-related tasks**, participants often preferred official documentation, with one participant noting how for Angular components, they would *go there and search for the template"* instead of using AI tools, as the official documentation provides *proper solution[s] that [are] 100% correct."* Another source participants mentioned using was Reddit, where they would seek *general guidance*, such as tool recommendations, from specific subreddits. We also found that when it comes to **tasks that require specific information, knowledge, or context**, or when working with specialized or niche libraries, our participants sought support directly from other developers or support teams on contract.

One participant highlighted the benefits of discussing their information needs with their colleagues, emphasizing that such interactions often lead to valuable, in-depth conversations and innovative ideas. This was especially the case when seeking information on coding practices and patterns for a large codebase, or working in niche spaces.

Policy & Practice

Regardless of the benefits, and perhaps due in part to the concerns, that come with using AI tools for seeking information, participants in our study highlighted several external factors that influence the impact and their use of AI tools in software development.

Participants noted that their usage of AI tools for information seeking is heavily influenced by the **adoption and promotion of AI within the organization**. Some organizations are proactive in adopting and promoting AI use, aiming to utilize these tools responsibly and in a controlled manner, while others feel *"it's not worth your time"*. However, this proactive stance can sometimes lead to an overshadowing of traditional engineering practices, resulting in less investment in non-AI related professional development.

> *"I'd say on the negative side, I feel like AI has taken over the industry so much that it's pushed out some traditional engineering. And so it's less motivating for employers to want to invest in learning a unique knowledge or unique tool... non technical people who run private companies [tend to] think that AI could just solve things. So there may not be as many professional development resources for anything non-AI related."[P17]*

4.2 Impact on Perceived Productivity (RQ2)

Despite the potential for new tooling to be disruptive to developer workflows, participants from our survey and interviews indicated AI tools may not be as counter-productive when information seeking. One participant emphasized this distinction, stating, *"The efficiency gained is worth the loss of flow. But for me, flow is less about thinking and more about implementing. The AI is doing the implementing, so the work I am doing is overcoming the roadblocks, not doing the easy work that flows."* For most of our survey respondents, AI tools at the very least have no impact on their workflow (109), where respondents reported that their workflow was about the same when using AI tools as when they do not. For some respondents, mostly those with fewer years of development experience, using AI tools for information seeking actually improves their flow (78). For those who felt AI tools improved their flow, the rationale was most often not having to look at as many sources, being provided step-by-step guidance, and accelerated learning.

Measuring Productivity
To better contextualize the perceived productivity reported by developers, we asked participants to describe what productivity meant to them and how they measured it. While **time savings** was the most common metric of productivity among our participants, our efforts uncovered other ways developers think about productivity. When considering time savings, this can be the time required to complete tasks or projects, debug and resolve issues, and make decisions. We also found that **output quantity** was another metric; this refers to the number of tickets or tasks completed per day or sprint, as well as publication output for research and development teams. One participant noted that they could *"get a ticket that has been scoped for several days or a week done in a half an hour"*. Another stated when they can close *"[more] tickets [than usual] in a day"* or *"[multiple] feature development and a couple of bug fixes"* then they know *"the tool is assisting [me] in the right way."*

Some participants also emphasized the role of **solution quality** in their productivity, which includes things like frequency of issues arising and the number of code review iterations required.

Project and Team Impact
Our participants highlighted the impact of using AI tools for information seeking beyond the individual developer. For some, there have been visible impacts to the projects they are working on when using AI tools. Participants cited AI assistants making it easier to parse large volumes of documentation to **help pinpoint and resolve issues faster** and providing code optimization strategies that speed up application. Some also discussed AI tools helping them clarify requirements and consider edge cases, leading to **faster sprint completion times** and more robust solutions. When discussing their own experiences, one participant described:

> *"I needed to handle a bulk of requests from customers' mobile devices. Initially, I assumed the number of requests wouldn't exceed a certain limit, but there was an instance where this assumption was proven wrong, leading to a significant surge. I consulted with GPT, and we implemented a quick fix that day... Since then, I regularly check with GPT for potential edge cases, which helps me anticipate issues and incorporate necessary adjustments proactively." [P6]*

On the flip side, participants discussed negative impacts AI tool use can have, particularly on software teams. Our participants report **reduced face-to-face interactions and organic knowledge sharing**, which can impact team cohesion and collaboration:

> *"We don't have as much synergy with each other... we're less incentivized to bounce ideas off each other or ask for help on concepts, and that organic conversation has gone missing. I work with a renowned scientist in that area, but I'm more likely to go to AI for help." [P11]*

Participants feel especially more comfortable using AI tools when they are less knowledgeable about something, as they may feel self-conscious about approaching someone more expert:

> *"We used to have different Slack channels for Python help, Excel, or web development. Those have mainly gone silent except for sharing bad AI answers. People prefer using AI instead of outing themselves for not knowing certain coding conventions or making silly errors." [P11]*

4.3 Impact on Skill Development (RQ3)

Using AI Tools to Learn
Many participants discussed how using AI tools when seeking information supports their learning and skill-building. Most often, participants mentioned the effectiveness of AI tools for **filling in knowledge gaps** and **providing practice problems** that support hands-on learning. For these participants, using AI tools in this way helps reinforce new concepts and improve problem-solving skills.

Participants also frequently used AI tools for learning best practices, citing them as valuable resources for **understanding industry standards** and improving coding techniques.

AI-driven information seeking can also support the **discovery and adoption of new solutions**. An overwhelming majority of survey respondents, including more experienced developers, reported using AI for learning new technologies (119).

Participants in our study found AI tools useful for **exploring innovative approaches and alternative methods** for solving programming challenges, where they noted being able to get acquainted with new technologies without the overhead of seeking and learning about each individually. One participant noted being *"more willing to use new tooling I don't understand, barely understand, or rarely work with."* AI tools have also played a role in building other participants' confidence in using new libraries or technologies, though some still reported discomfort in applying the knowledge gained from AI tools independently to practical scenarios. For one participant, this can be a product of overreliance, stating, *"if you keep looking at ChatGPT for more and more alternate solutions, it just makes you lose your confidence."*

While many interviews suggest usefulness of AI tools for learning, a high proportion of survey respondents indicated they experience a higher learning curve when learning about and *understanding* new technologies using AI. For some participants, AI tools are not suited for supporting their learning, despite their ability to provide personalized responses, as they often provide inadequate levels of detail. As stated by one participant:

> *"The information is not typically presented in a pedagogically or intelligently designed way, like, you have to know what question to ask to be able to get an answer. But there is no pedagogy. There's no there's no like "Oh, I can tell, based on this question that you're just now learning this, and let me teach you in a way that I expect you to be able to retain this information or to give you more thorough context or links to resources that you can investigate on your own that are like good verifiable links" or anything like that. So yeah, I would not use it as a primary source of learning."[P17]*

Impact on Skill Development
When discussing the ways in which they feel AI tools impact their skill-building, many participants cited appreciating the **ability to expand their knowledge and technical skillset more quickly** with AI tools, but expressed concerns about potential knowledge

gaps (88), overreliance (94), and poor problem-solving skills (87) resulting from AI use for learning.

One notable drawback that emerged from our interviews was the **decrease in creative problem-solving abilities**, leading to fundamental issues like reduced ability to code independently and diminished understanding of technologies.

> *"It's like when we were doing calculations on our own. But then calculators came in. So we don't calculate it in our mind anymore... I just don't think about it on my own anymore, because I am relying a lot on AI tools... It has greatly reduced my [ability to think independently]."[P7]*

This extends to **diminished learning of fundamental development skills** like command line and debugging, where developers *"just throw the error, or whatever problem,...to ChatGPT and it will fix it."* This problem is compounded by AI providing answers limited to specific questions asked, lacking comprehensive context. Some participants noted the challenge of not knowing how to formulate effective questions when learning something new: *"it only covers the range of questions I have in my mind...it is like giving an answer to your question, but not delivering a session on one topic."*

Interestingly, some participants reported that AI fostered **a different kind of creative and critical thinking skillset**. They described how correcting AI suggestions requires creative problem-solving:

> *"...when I'm done I approach GPT for practice... I'll ask GPT to give me [practice] problems... I would start thinking of [how to solve] them. And then I would ask it again to give me the solution when I'm done [figuring out] my own so that I can compare and think. It helps me think in a different way."[P2]*

Some even reported AI improving creativity by exposing them to new approaches: *"I think it promotes creativity. Because you may be introduced to new ways or new patterns of doing something...I think that it sometimes, by being too wordy, will like accidentally give people ideas that they otherwise wouldn't have... Yeah, I'd say it's like a net positive win in the creativity department"[P17].*

Survey respondents reported using AI tools for both learning about (114) and integrating (113) new technologies. However, from our interviews, we found there may be differences in the impact of using AI tools for learning about new technologies rather than integrating new technologies:

> *"I think AI tools can provide a temporary solution for overcoming a particular situation, but they can make it harder to truly understand the core knowledge or information that the technology or task involves. Since AI tools give us direct steps and solutions, we end up doing less research on our own, which can limit our knowledge and skills in the long run."[P1]*

Participants felt that while AI can expedite problem solving, it can also lead to a **superficial understanding of new technologies** if used for *integration*, which may be the reason why some participants reported feeling less confident and comfortable in using these technologies independently for learning. When discussing their use of AI tools for learning, one participant noted that AI *"generally gives us an overview or brief about it, but not the complete information we need."*

5 Discussion

Our findings indicate a shift in how developers are seeking information and building expertise in the age of AI assisted tooling. Below we discuss important insights into broader, existing concerns around the increasing integration of AI into software development.

5.1 The Evolution of Developer Information Seeking

Our research reveals a shift from goal-oriented to task-oriented learning approaches in AI-assisted development. While traditional goal-oriented learning emphasized building comprehensive understanding by exploring available resources in-depth, the current task-oriented paradigm focuses on immediate problem resolution using AI tools as on-demand information providers. This represents a significant shift from established patterns where developers built expertise through reading documentation and social learning (such as peer interactions and collaborative forums) before implementation [9, 39].

Our findings suggest that the timing and nature of information seeking may be shifting as well. Developers now typically proceed directly to implementation with AI support, seeking contextual information *reactively* when encountering specific obstacles. This just-in-time pattern is a contrast from traditional approaches where developers first built foundational knowledge through documentation and peer discussions. While this new approach accelerates immediate task completion, it introduces additional challenges. Another challenge lies in the validation of AI-generated solutions. Unlike traditional information sources that improve through community vetting and peer review, AI-generated content requires individual validation for each instance. This verification burden partially offsets the efficiency gains offered by AI assistance and shifts the responsibility of verification from the community to individual developers. We discuss potential directions for addressing these emerging challenges through developer tool support in Section 5.3

5.2 Developer Productivity and Learning Trade-offs

Developers traditionally build expertise through deliberate practice, pattern recognition, and incremental learning – all of which are well established principles from educational psychology [3, 8, 11, 15, 35]. Self-explanation, reflection, and social learning through mentorship and collaboration have also been crucial components in knowledge transfer and skill development [7].

Our research suggests significant trade-offs between immediate productivity gains and long-term expertise building in AI-assisted task completion. While our participants reported, and prior work suggests [14, 33, 38, 40], AI tools demonstrably increase short-term productivity through faster task completion and reduced blocked time, they also discussed the potential for leveraging AI tools for information seeking to simultaneously impede certain aspects of learning and skill development. We also found that this can lead

to decreased confidence in their ability to work with new tools or technologies without the support of AI assistants.

In the current landscape, access to AI-generated solutions is widespread which our findings suggest can reduce the struggle of being productive while learning. However, it can also potentially limit the development of key skills like problem solving and deeper understanding of technical concepts. Furthermore, while AI tools make information more readily available, the fragmented nature of AI-assisted learning may impede the integration of the low-level knowledge acquired into a coherent mental model of software systems and concepts that can be translated to other scenarios [32]. This creates complex trade-offs between immediate efficiency and sustained expertise development that warrant careful consideration as AI tools become increasingly integrated into development workflows.

5.3 Directions for AI-Assisted Developer Tools

Central to evolving as an engineer is the ability to build and retain expertise in technical concepts. While our findings emphasize potential trade-offs that may come with using AI-assisted tools for development tasks, they also suggest potential directions for next-generation AI-assisted development tools that better balance immediate assistance with long-term learning support:

Retrieval Augmented Development Environments: While current AI tools commonly used by developers often operate in isolation from project contexts, Retrieval-Augmented Generation (RAG) systems are a promising direction for improving both productivity and solution quality [26]. By grounding AI assistance in project documentation, codebase history, relevant forum discussions and technical specifications, RAG systems could help developers more effectively consolidate, leverage, and connect existing knowledge. This could be particularly valuable for tasks like navigating complex documentation, understanding API usage patterns, or debugging specific error cases that require synthesizing information from multiple sources. They could also help automate validation by connecting AI suggestions to trusted sources and providing clear provenance for generated solutions.

Adaptive Learning Systems: Tools need to evolve beyond static assistance to support different learning stages and developer growth. Prior work suggests that adaptive learning systems can be a useful model to replicate in the context of improving developer tools [23]. By creating more adaptive AI-assisted developer tools, these systems can better support learning and expertise building by recognizing a developer's progression from novice to expert and adjusting the depth and style of assistance accordingly. For instance, they might provide more comprehensive explanations for newcomers while offering more context-specific, advanced suggestions as expertise grows.

Task-Specific Fine-tuning: Rather than relying on general-purpose models, AI-assisted tools being used in the context of software development should be specifically fine-tuned for distinct software engineering tasks. This includes specialized models for code review, architectural decision support, security analysis, and learning assistance where each model is optimized for its specific use case and incorporates the most relevant best practices and knowledge.

Knowledge Integration and Sharing: As emphasized in prior efforts, software engineering is a collaborative activity where learning and expertise-building is often facilitated by peers [7]. To avoid AI-assisted tools replacing or interfering with knowledge sharing among teams and the broader community, these tools could actively facilitate and incentivize knowledge sharing and validation with others. This could include features for documenting AI-assisted solutions, sharing verified responses, and building team-specific knowledge bases that combine traditional documentation with validated AI-generated content.

6 Threats to Validity

Internal. Our study relies on self-reported data from surveys and interviews, which may be subject to memory bias. We asked participants to ground their response in past experiences to mitigate this threat.

External. Our sample was recruited through LinkedIn, X, and internal mailing lists, which might not be representative of the broader developer community. Developers who are not active on these platforms or in these networks might have different experiences with AI tools. To mitigate this, we also engaged with local developer communities and recruited participants through snowball sampling. Human-centered empirical studies are also prone to generalizability issues, due in part to concerns like sample size. However, the goal of our study is not to be generalizable, but rather transferable [16]. We ensure this by using a mixed-method approach where we supplement our survey findings with interviews. In our qualitative analysis, we followed best practices to ensure rigor. We did not report qualitative data using quantitative methods to prevent misinterpretation of our findings [30]. To further ensure the validity of our thematic analysis, we invited an external auditor to review our methodology and findings.

Construct. As with any survey or interview, there is potential for misinterpretations questions by participants. To mitigate this, we piloted our survey and interview protocols with multiple participants to ensure clarity and refined questions as needed. We also provided definitions for key terms to ensure participants had a consistent understanding of the concepts being discussed.

7 Conclusion

In this paper, we describe our efforts exploring how information-seeking behavior has evolved in the era of AI tools. Based on data collected from a survey and set of interviews, we report on the kinds of information developers use AI tools to seek, challenges that come with using AI tools for information seeking, and the impact this has on developer productivity and skill development. Our work provides novel insights and implications regarding the importance of foundational knowledge in effective AI tool use, the potential for AI tools use to increase productivity, and best practices for AI tools as learning aids in software development.

References

[1] [n. d.]. Methodology | 2024 Stack Overflow Developer Survey — survey.stackoverflow.co. https://survey.stackoverflow.co/2024/methodology. [Accessed 02-08-2024].

[2] Anonymous. 2025. Supplementary Materials. https://anonymous.4open.science/r/info-seeking-study-32DF/ Accessed: Feb 28, 2025.

[3] Richard C Atkinson and Richard M Shiffrin. 1968. Human memory. *A proposed system and its control processes* (1968).

[4] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.

[5] Andrew Begel and Beth Simon. 2008. Novice software developers, all over again. In *Proceedings of the fourth international workshop on computing education research*. 3–14.

[6] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2022. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (2022), 35–57.

[7] Samuel Boguslawski, Rowan Deer, and Mark G Dawson. 2024. Programming education and learner motivation in the age of generative AI: student and educator perspectives. *Information and Learning Sciences* (2024).

[8] Daniel Bor and Anil K Seth. 2012. Consciousness and the prefrontal parietal network: insights from attention, working memory, and chunking. *Frontiers in psychology* 3 (2012), 63.

[9] Joel Brandt, Philip J Guo, Joel Lewenstein, Mira Dontcheva, and Scott R Klemmer. 2009. Two studies of opportunistic programming: interleaving web foraging, learning, and writing code. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1589–1598.

[10] Silvia Breu, Rahul Premraj, Jonathan Sillito, and Thomas Zimmermann. 2010. Information needs in bug reports: improving cooperation between developers and users. In *Proceedings of the 2010 ACM conference on Computer supported cooperative work*. 301–310.

[11] Guillermo Campitelli and Fernand Gobet. 2011. Deliberate practice: Necessary but not sufficient. *Current directions in psychological science* 20, 5 (2011), 280–285.

[12] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. "It would work for me too": How Online Communities Shape Software Developers' Trust in AI-Powered Code Generation Tools. *ACM Transactions on Interactive Intelligent Systems* 14, 2 (2024), 1–39.

[13] Saikrishna Chinthapatla. 2024. Unleashing the Future: A Deep Dive into AI-Enhanced Productivity for Developers. *Journal Homepage: http://www. ijmra. us* 13, 03 (2024).

[14] Mariana Coutinho, Lorena Marques, Anderson Santos, Marcio Dahia, Cesar França, and Ronnie de Souza Santos. 2024. The Role of Generative AI in Software Development Productivity: A Pilot Case Study. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 131–138.

[15] Nelson Cowan, Zhijian Chen, and Jeffrey N Rouder. 2004. Constant capacity in an immediate serial-recall task: A logical sequel to Miller (1956). *Psychological science* 15, 9 (2004), 634–640.

[16] Ben K Daniel. 2019. What constitutes a good qualitative research study? Fundamental dimensions and indicators of rigour in qualitative research: The TACT framework. In *Proceedings of the European conference of research methods for business & management studies*. 101–108.

[17] Denae Ford, Tom Zimmermann, Christian Bird, and Nachiappan Nagappan. 2017. Characterizing software engineering work with personas based on knowledge worker actions. In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 394–403.

[18] Thomas Fritz and Gail C Murphy. 2010. Using information fragments to answer the questions developers ask. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*. 175–184.

[19] Márcio Kuroki Gonçalves, Cleidson RB de Souza, and Víctor M González. 2011. Collaboration, Information Seeking and Communication: An Observational Study of Software Developers' Work Practices. *J. Univers. Comput. Sci.* 17, 14 (2011), 1913–1930.

[20] Marybec Griffin, Richard J Martino, Caleb LoSchiavo, Camilla Comer-Carruthers, Kristen D Krause, Christopher B Stults, and Perry N Halkitis. 2021. Ensuring survey research data integrity in the era of internet bots. *Quality & quantity* (2021), 1–12.

[21] Greg Guest, Arwen Bunce, and Laura Johnson. 2006. How many interviews are enough? An experiment with data saturation and variability. *Field methods* 18, 1 (2006), 59–82.

[22] Brittany Johnson, Christian Bird, Denae Ford, Nicole Forsgren, and Thomas Zimmermann. 2023. Make your tools sparkle with trust: The PICSE framework for trust in software tools. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 409–419.

[23] Brittany Johnson, Rahul Pandita, Emerson Murphy-Hill, and Sarah Heckman. 2015. Bespoke tools: adapted to the concepts developers know. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 878–881.

[24] Amy J Ko, Robert DeLine, and Gina Venolia. 2007. Information needs in collocated software development teams. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 344–353.

[25] Thomas D LaToza and Brad A Myers. 2010. Hard-to-answer questions about code. In *Evaluation and usability of programming languages and tools*. 1–6.

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[27] Paul Luo Li, Amy J Ko, and Andrew Begel. 2020. What distinguishes great software engineers? *Empirical Software Engineering* 25 (2020), 322–352.

[28] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[29] Mingwei Liu, Xin Peng, Andrian Marcus, Shuangshuang Xing, Christoph Treude, and Chengyuan Zhao. 2021. Api-related developer information needs in stack overflow. *IEEE Transactions on Software Engineering* 48, 11 (2021), 4485–4500.

[30] Joseph A Maxwell. 2010. Using numbers in qualitative research. *Qualitative inquiry* 16, 6 (2010), 475–482.

[31] André N Meyer, Earl T Barr, Christian Bird, and Thomas Zimmermann. 2019. Today was a good day: The daily life of software developers. *IEEE Transactions on Software Engineering* 47, 5 (2019), 863–880.

[32] Lynn Carol Miller and Stephen J Read. 2013. On the coherence of mental models of persons and relationships: A knowledge structure approach. In *Cognition in close relationships*. Psychology Press, 69–99.

[33] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).

[34] Shaun Phillips, Guenther Ruhe, and Jonathan Sillito. 2012. Information needs for integration decisions in the release process of large-scale parallel development. In *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 1371–1380.

[35] Henry L Roediger and Kathleen B McDermott. 1995. Creating false memories: Remembering words not presented in lists. *Journal of experimental psychology: Learning, Memory, and Cognition* 21, 4 (1995), 803.

[36] Daniel Russo. 2024. Navigating the complexity of generative ai adoption in software engineering. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–50.

[37] Margaret-Anne Storey and Alexey Zagalsky. 2016. Disrupting developer productivity one bot at a time. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*. 928–931.

[38] Tran Minh Tung. 2024. Opening Up The Workplace: The Way Ai Tools Are Changing Productivity. *Educational Administration: Theory and Practice* 30, 3 (2024), 480–491.

[39] Majid Zamiri and Ali Esmaeili. 2024. Methods and technologies for supporting knowledge sharing within learning communities: A systematic literature review. *Administrative Sciences* 14, 1 (2024), 17.

[40] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.